

7.Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```
bindu@binduaree:~$ nano skill7.c
bindu@binduaree:~$ gcc skill7.c -o skill7
bindu@binduaree:~$ ./skill7
PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinux.2.7.11.0_x64__8wekyb3d86fe6a6/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program File
s (x86)/Intel/Intel(R) Management Engine Components/DAL:/mnt/c/Program Files/Intel/Intel(R) Management Engine Components/DAL:/mnt/c/Users/User/Downloads/gcc-15.2.0-pdb-16.3.90.20250811-binutils-2.45-mingw-w64
-v13.0.0-ucrt/bin:/mnt/c/Program Files/nodejs:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/MongoDB/Server/8.3/bin:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/User/AppData/Local/Programs
/Python/Launcher:/mnt/c/Users/User/.local/bin:/mnt/c/Users/User/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/User/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/User/AppData/Local/Python/bin:/
mnt/c/Users/User/AppData/Roaming/npm:/mnt/c/Users/User/AppData/Local/Programs/cursor/resources/app/bin:/mnt/c/Users/User/AppData/Local/Programs/Antigravity IDE/bin:/mnt/c/Users/User/AppData/Local/Programs/mon
gosh:/snap/bin

Parent waiting for child...
Child Process Started
Child PID: 957
Hello save      hello.c.save      practical5b.c
HelloFromParent.c input.txt      priority.c
MultiChild      input.txt      priority.c.save
MultiChild.c    input.txt.save processid
Week1_1         input.txt.save.1 processid.c
Week1_1.c       input.txt.save.2 signal.c
Week1_2         linux          simplecommandLoop
Week1_2.c       linux.c        simplecommandLoop.c
childusingfork  multiplechild  sff
childusingfork.c multiplechild.c sff.c
client          myfifo         sff.c.save
client.c        myfifo.c       skill3a
concurrently    myprocessid    skill3a.c
concurrently.c  myprocessid.c skill3b
doublefork      myprocessid.c.save skill3b.c
doublefork.c    orphanprocess  skill6
fcfs            orphanprocess.c skill6.c
fcfs.c          output.txt     skill7
fifo            parent         skill7.c
fifo.c          parent.c       socket
fifo_read       parent.save    socket.c
fifo_read.c     parentchild    username
fifo_write      parentchild.c  username.c
fifo_write.c    parentwaitsforchild wait
file_write.c    parentwaitsforchild.c wait.c
fork            posix.c.save   welcome
fork.c          practical1     welcome.c
fork1.c         practical1.c   zombie
fork1.c         practical2     zombie.c
hello           practical2.c
hello.c         practical5b
Child exited with status: 0
bindu@binduaree:~$
```

```
waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
}

return 0;
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

^C Location

^_ Go To Line

^M-U Undo

^M-E Redo

GNU nano 8.7.1 skill7.c

bindu@Bindusree: ~

GNU nano 8.7.1

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

char *path = getenv("PATH");

if (path == NULL) {

printf("PATH variable not found.\n");

return 1;

}

printf("PATH Variable:\n%s\n", path);

pid_t pid = fork();

if (pid < 0) {

perror("fork failed");

return 1;

}

if (pid == 0) {

// Child process

printf("Child Process Started\n");

printf("Child PID: %d\n", getpid());

// Execute ls

execlp("ls", "ls", NULL);

// Runs only if exec fails

perror("exec failed");

exit(1);

}

else {

// Parent process

int status;

printf("Parent waiting for child...\n");

waitpid(pid, &status, 0);

if (WIFEXITED(status)) {

printf("Child exited with status: %d\n",

WEXITSTATUS(status));

}

}